BetArena - Contrat API REST v2.0

> **Cours** : IHDCB336 - Architecture des Applications Web
> **Groupe n°4** : Jérémy Dehaye - Timothy Lefebvre - Sacha Pauwels
> **Date de remise** : 22 février 2026
> **Échéance finale** : 31 mai 2026
> **Stack** : Spring Boot 3 - PostgreSQL - Vite TypeScript
> **Rédacteur contrat API**: Sacha PAUWELS
> **Révision** : v2.0 - Avril 2026

---

1. Généralités

1.1 Base URL

```
http://localhost:8080/api
```

En production, remplacer par le domaine du serveur déployé.

1.2 Format des échanges

- **Content-Type** : `application/json` pour toutes les requêtes et réponses.
- **Dates** : format ISO 8601 UTC - `2026-04-01T18:00:00Z`.

1.3 Authentification

L'authentification repose sur des **tokens JWT (Bearer)**. Après connexion ou inscription, le client reçoit un token à inclure dans chaque requête protégée :

```
Authorization: Bearer <token>
```

Les routes marquées `PUBLIC` ne nécessitent pas de token.

1.4 Convention des rôles

Badge	Signification
`PUBLIC`	Aucune authentification requise
`ANY`	Tout utilisateur connecté, quel que soit le rôle
`BETTOR`	Utilisateur connecté avec le rôle parieur
`BOOKMAKER`	Administrateur - accès complet à la gestion des événements

1.5 Structure d'erreur standard

Toute réponse d'erreur adopte la structure suivante :

```
{
  "status": 400,
  "error": "Bad Request",
  "message": "Solde insuffisant pour effectuer ce pari.",
  "timestamp": "2026-03-30T10:00:00Z",
  "path": "/api/bets"
}
```

1.6 Codes HTTP utilisés

Code	Signification
`200 OK`	Requête traitée avec succès.
`201 Created`	Ressource créée avec succès (inscription, pari, événement).
`400 Bad Request`	Données invalides ou contrainte métier non respectée (solde insuffisant, montant négatif…).
`401 Unauthorized`	Token absent, expiré ou invalide.
`403 Forbidden`	Token valide mais rôle insuffisant.
`404 Not Found`	Ressource introuvable.
`409 Conflict`	Conflit d'état : pari sur un événement fermé, résolution d'un événement déjà résolu…
`422 Unprocessable Entity`	Validation métier échouée (cote ≤ 1.0, nombre d'issues insuffisant…).
`500 Internal Server Error`	Erreur serveur non anticipée.

2. Authentification `/auth`

2.1 Inscription

```
POST /api/auth/register
```

> `PUBLIC`

Request Body

Champ	Type	Requis	Description
`username`	string	Oui	Nom d'utilisateur unique (3–30 caractères).

Champ	Type	Requis	Description
email	string	Oui	Adresse e-mail valide et unique.
password	string	Oui	Mot de passe (8 caractères minimum).

Réponse 201 Created

```
{
  "id": 1,
  "username": "johndoe",
  "email": "john@example.com",
  "role": "BETTOR",
  "balance": 1000.0,
  "token": "<jwt>",
  "createdAt": "2026-03-30T10:00:00Z"
}
```

> Le capital de départ (1 000 points) est attribué automatiquement par le serveur. Il n'est pas transmis par le client.

2.2 Connexion

```
POST /api/auth/login
```

> PUBLIC

Request Body

Champ	Type	Requis	Description
email	string	Oui	Adresse e-mail enregistrée.
password	string	Oui	Mot de passe en clair (transmis via HTTPS).

Réponse 200 OK

```
{
  "token": "<jwt>",
  "user": {
    "id": 1,
    "username": "johndoe",
    "role": "BETTOR",
    "balance": 1000.0
  }
}
```

2.3 Profil courant

```
GET /api/auth/me
```

> ANY

Réponse `200 OK`

```
{
  "id": 1,
  "username": "johndoe",
  "email": "john@example.com",
  "role": "BETTOR",
  "balance": 850.0,
  "createdAt": "2026-03-01T09:00:00Z"
}
```

3. Événements `/events`

3.1 Lister les événements

```
GET /api/events
```

> ANY

Query Parameters

Paramètre	Type	Requis	Description
`status`	string	Non	Filtre par état : `OPEN`, `CLOSED`, `RESOLVED`. Défaut : `OPEN`.
`category`	string	Non	Filtre par catégorie (ex. `SPORT`, `POLITIQUE`, `INTERNE`).
`page`	integer	Non	Numéro de page (0-indexé). Défaut : `0`.
`size`	integer	Non	Nombre d'éléments par page. Défaut : `15`, max : `50`.
`sort`	string	Non	Tri : `closeAt,asc` | `closeAt,desc` | `createdAt,desc`.

Réponse `200 OK`

```
{
  "content": [
    {
      "id": 1,
```

```
      "title": "Vainqueur du Championnat de Belgique 2026",
      "description": "Quelle équipe remportera le titre cette saison ?",
      "category": "SPORT",
      "closeAt": "2026-05-15T20:00:00Z",
      "status": "OPEN",
      "totalVolume": 3200.0,
      "createdAt": "2026-03-01T10:00:00Z",
      "outcomes": [
        { "id": 1, "label": "Club Bruges", "odds": 1.75, "totalBets":
1800.0 },
        { "id": 2, "label": "Union SG", "odds": 2.4, "totalBets": 900.0 },
        { "id": 3, "label": "Anderlecht", "odds": 3.1, "totalBets": 500.0 }
      ]
    }
  ],
  "totalElements": 42,
  "totalPages": 3,
  "page": 0,
  "size": 15
}
```

3.2 Détail d'un événement

```
GET /api/events/{id}
```

> ANY

Path Variable

Paramètre	Type	Requis	Description
id	long	Oui	Identifiant numérique de l'événement.

Réponse `200 OK` - objet événement complet (même structure que dans la liste).
Réponse `404 Not Found` si l'événement n'existe pas.

3.3 Créer un événement

```
POST /api/events
```

> BOOKMAKER

Request Body

Champ	Type	Requis	Description
`title`	string	Oui	Intitulé de l'événement (max 200 caractères).
`description`	string	Non	Description détaillée (max 2000 caractères).
`category`	string	Non	Catégorie libre (`SPORT`, `POLITIQUE`, `CULTURE`, `INTERNE`...).
`closeAt`	datetime	Oui	Date/heure de clôture des paris - doit être future.
`outcomes`	array	Oui	Liste d'au moins 2 issues possibles.
`outcomes[].label`	string	Oui	Libellé de l'issue.
`outcomes[].odds`	number	Oui	Cote associée (> 1.0).

Exemple

```
{
  "title": "Match Belgique vs France – Euro 2026",
  "category": "SPORT",
  "closeAt": "2026-06-14T20:45:00Z",
  "outcomes": [
    { "label": "Victoire Belgique", "odds": 2.8 },
    { "label": "Match nul", "odds": 3.2 },
    { "label": "Victoire France", "odds": 2.1 }
  ]
}
```

Réponse `201 Created` - objet événement créé avec `id` et `status: OPEN`.

3.4 Clôturer un événement

```
PATCH /api/events/{id}/close
```

> `BOOKMAKER`

Pas de corps de requête. L'événement doit être dans l'état `OPEN`.

Réponse `200 OK` - objet événement mis à jour avec `status: CLOSED`.
Réponse `409 Conflict` si l'événement est déjà fermé ou résolu.

> La clôture automatique (dépassement de `closeAt`) est gérée côté serveur via un scheduler `@Scheduled`. Cet endpoint permet une clôture manuelle anticipée par le bookmaker.

3.5 Résoudre un événement

```
PATCH /api/events/{id}/resolve
```

> `BOOKMAKER`

Request Body

Champ	Type	Requis	Description
`winningOutcomeId`	long	Oui	Identifiant de l'issue déclarée gagnante.

```
{
  "winningOutcomeId": 1
}
```

Réponse `200 OK`

```
{
  "id": 1,
  "status": "RESOLVED",
  "winningOutcome": { "id": 1, "label": "Club Bruges", "odds": 1.75 },
  "totalDistributed": 3150.0,
  "resolvedAt": "2026-05-16T09:30:00Z"
}
```

> La distribution est **atomique** : le serveur calcule pour chaque parieur gagnant `gain = mise × cote`, crédite les soldes et marque les paris `WON`/`LOST` en une seule transaction base de données.

Réponse `409 Conflict` si l'événement est déjà résolu ou encore `OPEN`.
Réponse `404 Not Found` si `winningOutcomeId` n'appartient pas à l'événement.

---

4. Paris `/bets`

4.1 Placer un pari

```
POST /api/bets
```

> `BETTOR`

Request Body

Champ	Type	Requis	Description
`outcomeId`	long	Oui	Identifiant de l'issue ciblée.

Champ	Type	Requis	Description
`amount`	number	Oui	Montant à miser en points virtuels (> 0).

```
{
  "outcomeId": 1,
  "amount": 150.0
}
```

Réponse `201 Created`

```
{
  "id": 42,
  "eventId": 1,
  "eventTitle": "Vainqueur du Championnat de Belgique 2026",
  "outcomeId": 1,
  "outcomeLabel": "Club Bruges",
  "odds": 1.75,
  "amount": 150.0,
  "potentialGain": 262.5,
  "status": "PENDING",
  "placedAt": "2026-03-30T11:00:00Z",
  "newBalance": 850.0
}
```

> `newBalance` reflète le solde du parieur après débit - évite au frontend de refaire un `GET /auth/me` pour rafraîchir l'affichage.

Erreurs spécifiques

Code	Signification
`400 Bad Request`	`amount` ≤ 0 ou solde insuffisant.
`409 Conflict`	L'événement est `CLOSED` ou `RESOLVED`.
`404 Not Found`	`outcomeId` introuvable.

4.2 Mes paris

```
GET /api/bets/me
```

> `BETTOR`

Query Parameters

Paramètre	Type	Requis	Description
`status`	string	Non	Filtre : `PENDING`, `WON`, `LOST`. Défaut : tous.
`page`	integer	Non	Numéro de page (0-indexé). Défaut : `0`.
`size`	integer	Non	Taille de page. Défaut : `20`.

Réponse `200 OK` - liste paginée de paris (même structure que le `POST`, sans `newBalance`).

---

5. Utilisateurs `/users`

5.1 Leaderboard

```
GET /api/users/leaderboard
```

> `PUBLIC`

Query Parameters

Paramètre	Type	Requis	Description
`limit`	integer	Non	Nombre de joueurs à retourner. Défaut : `10`, max : `50`.

Réponse `200 OK`

```
[
  { "rank": 1, "username": "alice", "balance": 5240.0 },
  { "rank": 2, "username": "bob", "balance": 3980.0 },
  { "rank": 3, "username": "charlie", "balance": 3210.0 }
]
```

> Seuls le `username` et le `balance` sont exposés publiquement. Aucune donnée sensible (email, mot de passe) n'est incluse.

---

6. Administration `/admin`

> Tous les endpoints de cette section sont réservés au rôle `BOOKMAKER`. Une requête avec un token `BETTOR` retourne systématiquement `403 Forbidden`.

6.1 Tous les événements

```
GET /api/admin/events
```

> `BOOKMAKER`

Query Parameters

Paramètre	Type	Requis	Description
`status`	string	Non	Filtre par état : `OPEN`, `CLOSED`, `RESOLVED`.
`category`	string	Non	Filtre par catégorie.
`page`	integer	Non	Numéro de page. Défaut : `0`.
`size`	integer	Non	Taille de page. Défaut : `20`.

Réponse `200 OK` - liste paginée d'événements enrichis avec le volume total de mises par issue.

6.2 Statistiques globales

```
GET /api/admin/stats
```

> `BOOKMAKER`

Réponse `200 OK`

```
{
  "totalUsers": 156,
  "totalEvents": 48,
  "openEvents": 12,
  "resolvedEvents": 30,
  "totalBetsVolume": 87450.0,
  "totalBetsCount": 1240,
  "totalDistributed": 61200.0
}
```

---

7. Récapitulatif des endpoints

Méthode	Endpoint	Rôle	Description
`POST`	`/api/auth/register`	`PUBLIC`	Inscription
`POST`	`/api/auth/login`	`PUBLIC`	Connexion
`GET`	`/api/auth/me`	`ANY`	Profil courant
`GET`	`/api/events`	`ANY`	Lister les événements
`GET`	`/api/events/{id}`	`ANY`	Détail d'un événement
`POST`	`/api/events`	`BOOKMAKER`	Créer un événement
`PATCH`	`/api/events/{id}/close`	`BOOKMAKER`	Clôturer un événement

Méthode	Endpoint	Rôle	Description
PATCH	/api/events/{id}/resolve	BOOKMAKER	Résoudre un événement
POST	/api/bets	BETTOR	Placer un pari
GET	/api/bets/me	BETTOR	Mes paris
GET	/api/users/leaderboard	PUBLIC	Classement public
GET	/api/admin/events	BOOKMAKER	Tous les événements
GET	/api/admin/stats	BOOKMAKER	Statistiques globales

---

8. Modèles de données

Aperçu des entités principales attendues côté Spring Boot / PostgreSQL.

8.1 User

```
id          BIGINT        PK AUTO_INCREMENT
username    VARCHAR(30)   NOT NULL UNIQUE
email       VARCHAR(255)  NOT NULL UNIQUE
password    VARCHAR(255)  NOT NULL          -- bcrypt hash
role        ENUM          NOT NULL          -- BETTOR | BOOKMAKER
balance     DECIMAL(10,2) NOT NULL DEFAULT 1000.00
created_at  TIMESTAMP     NOT NULL DEFAULT NOW()
```

8.2 Event

```
id          BIGINT        PK AUTO_INCREMENT
title       VARCHAR(200)  NOT NULL
description TEXT
category    VARCHAR(50)
close_at    TIMESTAMP     NOT NULL
status      ENUM          NOT NULL          -- OPEN | CLOSED | RESOLVED
created_by  BIGINT        FK → users(id)
created_at  TIMESTAMP     NOT NULL DEFAULT NOW()
resolved_at TIMESTAMP
```

8.3 Outcome (issue)

```
id        BIGINT        PK AUTO_INCREMENT
event_id  BIGINT        FK → events(id) NOT NULL
label     VARCHAR(100)  NOT NULL
odds      DECIMAL(6,2)  NOT NULL CHECK (odds > 1.0)
is_winner BOOLEAN       DEFAULT FALSE
```

8.4 Bet

```
id              BIGINT         PK AUTO_INCREMENT
user_id         BIGINT         FK → users(id)    NOT NULL
outcome_id      BIGINT         FK → outcomes(id) NOT NULL
amount          DECIMAL(10,2) NOT NULL CHECK (amount > 0)
potential_gain DECIMAL(10,2) NOT NULL          -- amount * odds
status          ENUM           NOT NULL          -- PENDING | WON | LOST
placed_at       TIMESTAMP      NOT NULL DEFAULT NOW()
```